

Securing the AI/ML Supply Chain (OWASP LLM03)

Every model, dataset, embedding index, package, and MCP server you pull in is a dependency that can execute code or shape behavior in your environment. OWASP LLM03 (Supply Chain) is consistently among the highest-impact LLM risks because a single compromised artifact bypasses every downstream guardrail.

Threats

- **Malicious / mutable dependencies.** A package installed from a VCS ref or an unpinned version can change under you (typosquatting, dependency confusion, account takeover of a maintainer).
- **Tampered model artifacts.** A backdoored model or LoRA adapter can carry a hidden trigger phrase that flips behavior (MITRE ATLAS AML.T0018). Unsafe formats (`pickle`-based checkpoints) can execute code on load.
- **Poisoned datasets / indexes.** Training, fine-tune, or RAG data sourced without integrity controls can carry planted facts or latent backdoors (LLM04/LLM08).
- **Untrusted MCP servers.** A third-party server runs code with your privileges and can poison tool descriptions (see `mcp-architecture-review.md`).

Controls

1. **Pin and lock dependencies.** Exact versions plus a hash-locked lockfile. No VCS/URL installs in production images. `airte.supply_chain.audit_requirements` flags unpinned and URL/VCS dependencies in CI.
2. **Verify model provenance.** Require an immutable revision/digest, a recorded trusted source, a license, and a hash before loading. `airte.supply_chain.audit_model_card` enforces this on a model card/manifest.
3. **Prefer safe formats.** Load models from `safetensors` rather than `pickle`-based checkpoints; never unpickle untrusted artifacts.
4. **Sign and verify data.** HMAC-sign records on ingest and verify before they enter a prompt or index (`airte.data_pipeline.provenance`) so tampering is detected.
5. **Behavioral backdoor probes.** Run the `supply_chain` red-team suite (`airte.redteam.supply_chain`) to test for hidden trigger phrases / override modes.
6. **Vet MCP servers and plugins** before onboarding; review tool descriptions as code.

In CI

```
from airte.supply_chain import audit_requirements, audit_model_card
deps = audit_requirements(open("requirements.txt").read())
model = audit_model_card({"name": "clf", "source": "hub", "sha256": "...",
                          "trusted": True, "license": "apache-2.0"})
```

```
assert not deps and not model, "supply-chain audit failed"
```

AI Bill of Materials (AI-BOM)

Export a **CycloneDX 1.6** AI-BOM that inventories both software dependencies and ML models (as `machine-learning-model` components with `modelCards`, hashes, and provenance properties). Feed it to SCA/vulnerability tooling and attach it to releases so consumers can audit your AI supply chain.

```
python -m airte.supply_chain.aibom --name my-app --version 1.0 \
  --requirements requirements.txt \
  --model "clf:1.0:hf:<sha256>:apache-2.0" -o aibom.json
```

Models without an immutable digest are tagged with an `ai:provenance` warning property so the gap is visible in the BOM.

Run the audit on every commit with the bundled pre-commit hook (`airte-supply-audit`), which blocks commits that introduce unpinned or URL/VCS dependencies.

VEX: is it actually exploitable?

Attach a **CycloneDX VEX** layer (`airte.supply_chain.vex`) to the AI-BOM to state, per advisory, whether each component is exploitable in *your* product (`not_affected` + justification, `exploitable`, `in_triage`, `false_positive`, ...). Affected refs are validated against the BOM so statements can't dangle, and `exploitable()` filters to the statements that actually need action. This is what lets a security team triage a flood of dependency CVEs down to the few that matter.

Combine the static audit (artifacts), the AI-BOM (inventory), the VEX layer (exploitability), the behavioral suite (runtime), and data signing (integrity) for layered coverage across the whole dependency chain.